Unit 4

Static Keyword in C++

The static keyword in C++ has different meanings when used with different types. In this article, we will learn about the static keyword in C++ along with its various uses.
In C++, a static keyword can be used in the following context:

Static Variables in a Function

In a function, when a variable is declared as static, space for **it gets allocated for the lifetime of the program**. Even if the function is called multiple times, space for the static variable is allocated only once and the value of the variable in the previous call gets carried through the next function call.

Let's take a look at an example:

```
#include <iostream.h>
using namespace std;

void f() {

    // Static variable
    static int count = 0;

    // The value will be updated and carried over
    // to the next function call
    count++;
    cout << count << " ";
}

int main() {

    // Calling function f() 5 times
    for (int i = 0; i < 5; i++)
        f();

    return 0;
}
```

Output
1 2 3 4 5

You can see in the above program that the variable count is declared static. So, its value is carried through the function calls. The variable count is not getting initialized every time the function is called. As a side note, Java doesn't allow static local variables in functions.

Applications

The static variables in a function have the following applications:

- Return local variable address from the function.
- Useful for implementing coroutines in C++ or any other application where the previous state of function needs to be stored.
- Memoization in recursive calls.

Static Data Member in a Class

As the variables declared as static are initialized only once as they are allocated space in separate static storage so, the static member variables in a class are shared by the objects. There cannot be multiple copies of the same static variables for different objects. Also because of this reason static variables cannot be initialized using constructors.

Let's take a look at an example:

```
#include <iostream>
using namespace std;

class GfG {
public:

   // Static data member
   static int i;

   GfG(){
      // Do nothing
   };
};

int main() {
   GfG obj1;
```

```
    GfG obj2;
    obj1.i = 2;
    obj2.i = 3;

    // Prints value of i
    cout << obj1.i << " " << obj2.i;
}
```

Output

```
undefined reference to `GfG::i'
collect2: error: ld returned 1 exit status
```

Explanation: You can see in the above program that we have tried to create multiple copies of the static variable i for multiple objects. But this didn't happen.
So, a static variable inside a class should be initialized explicitly by the user using the class name and scope resolution operator outside the class as shown below:

```
#include <iostream>
using namespace std;

class GfG {
public:

    // Static data member
    static int i;

    GfG(){
        // Do nothing
    };
};

// Static member inintialization
int GfG::i = 1;

int main() {

    // Prints value of i
    cout << GfG::i;
}
```

Output
1
Explanation: We were able to access the static variable when is was initialized globally outside the class. Moreover, we can access the static data member without creating the object of the class.

Applications

The static data members can be used to implement the following:

- Counting Objects of a Class
- Store and share configuration or settings globally.
- Tracking Shared Resources
- Regulate or limit operations performed by multiple objects.
- Ensure a class has only one instance by using static members.

Static Member Functions in a Class

Just like the static data members or static variables inside the class, static member functions also do not depend on the object of the class. We are allowed to invoke a static member function using the object and the '.' operator but it is recommended to invoke the static members using the class name and the scope resolution operator. Static member functions are allowed to access only the static data members or other static member functions, they cannot access the non-static data members or member functions of the class.

Let's take a look at an example:

```
#include <iostream>
using namespace std;

class GfG {
public:

    // Static member function
    static void printMsg() { cout << "Welcome to GfG!"; }
};

int main() {

    // Invoking a static member function
```

```
    GfG::printMsg();
}
```

Output
Welcome to GfG!

Applications

- Accessing Static Member Variables
- Implement helper functions that do not depend on specific instances.
- Singleton Pattern Implementation
- Factory Methods to create and return objects without requiring an instance of the class.
- Logging and Debugging

Global Static Variable

A global static variable in C++ is a static variable declared outside of any class or function. Unlike regular global variables, a global static variable has internal linkage, meaning it is accessible only within the file where it is defined. This ensures that its scope is limited to the current translation unit, preventing conflicts with variables in other files that may have the same name.

Let's take a look at an example:

```
#include <iostream>
using namespace std;

// Global static variable
static int count = 0;

void increment() {
    count++;
    cout << count << " ";
}

int main() {
    increment();
    increment();
    return 0;
}
```

Output
1 2

Applications

The global static variables have the following uses in C++:

- Limiting variable scope to a file to prevent conflicts by ensuring the variable is accessible only within the file.
- Global counters or flags.
- Store settings or values that are specific to the functionality implemented in a single file.
- Use for shared resources in scenarios where frequent initialization and destruction can be avoided.
- Shared state across functions in a file.

Const keyword in C++

The const keyword stands for constant. It is used to declare variables, objects, pointers, function parameters, return values, and class methods as read-only, meaning their value cannot be changed after initialization.

const variable must be initialized at the time of declaration.
Once initialized, its value cannot be modified or reassigned anywhere in the program.
how_to_declare_constants

```
#include <iostream>
using namespace std;

int main()
{

   const int y = 10;
   cout << y;

   return 0;
}
```

Output
10

Const Data Member

A const data member is a class variable whose value cannot be changed after it is initialized. Once set, it remains constant for the entire lifetime of the object.

Key Points

- Declared using the keyword const.
- Must be initialized when the object is created.
- Cannot be modified anywhere else in the program.
- Initialization is done using a constructor initializer list (not inside the constructor body).

Syntax Example

```
#include <iostream>
using namespace std;

class Student {
   const int rollNo;   // const data member

public:
   Student(int r) : rollNo(r) {   // initialization using initializer list
   }

   void display() {
      cout << "Roll Number: " << rollNo << endl;
   }
};

int main() {
   Student s1(101);
   s1.display();
   return 0;
}
```

output

Roll Number : 101

Const Member Functions

Class objects can be declared const, preventing modification of their data members. They can only call const member functions, must be initialized at declaration (usually via a constructor), and const functions can be invoked by both const and non-const objects.

There are two ways to declare a constant function:

Ordinary Const Function: Declaring a non-member function as const has no real effect; void foo() const is invalid outside a class.

Const Member Function: In a class, a member function declared const cannot modify the object's data members.

```
// C++ program to demonstrate the
// constant function
#include <iostream>
using namespace std;

// Class Test
class Test
{
   int value;

  public:
   // Constructor
   Test(int v = 0)
   {
      value = v;
   }

   // We get compiler error if we
   // add a line like "value = 100;"
   // in this function.
   int getValue() const
   {
      return value;
   }

   // a nonconst function trying to modify value
   void setValue(int val)
   {
```

```
        value = val;
    }
};

// Driver Code
int main()
{
    // Object of the class T
    Test t(20);

    // non-const object invoking const function, no error
    cout << t.getValue() << endl;

    // const object
    const Test t_const(10);

    // const object invoking const function, no error
    cout << t_const.getValue() << endl;

    // const object invoking non-const function, CTE
    // t_const.setValue(15);

    // non-const object invoking non-const function, no
    // error
    t.setValue(12);

    cout << t.getValue() << endl;

    return 0;
}
```

Output

```
20
10
12
```

Comparision Between Static and Const data member and member function:

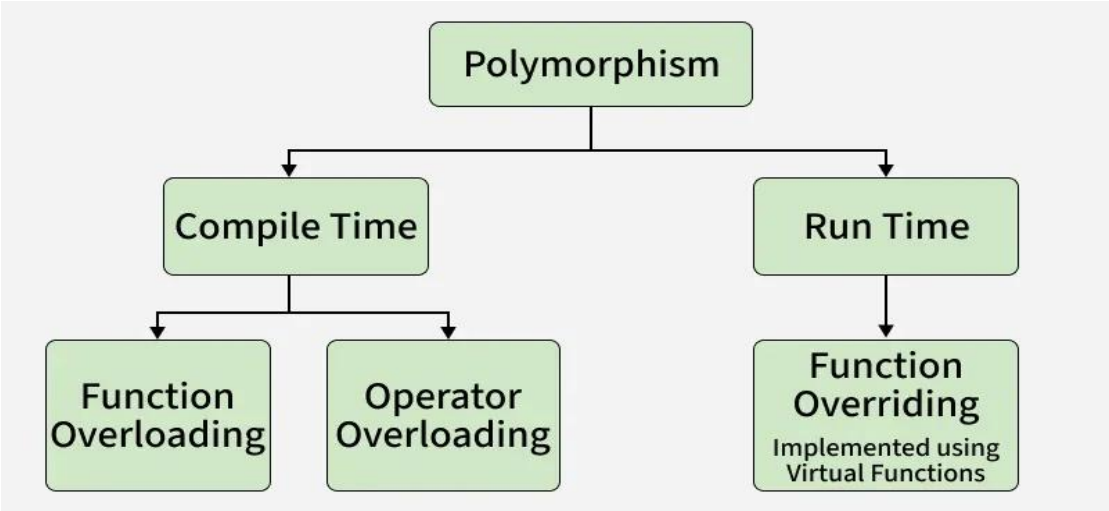
Feature	Static Data Member	Static Member Function	Const Data Member	Const Member Function
Keyword	static	static	const	const
Belongs to	Class	Class	Object	Object
Memory	One copy shared	No object memory	Separate per object	No extra memory
Initialization	Outside class	Not needed	Constructor initializer list	Not needed
Can modify data	Yes	Only static data	No	No
Access object data	Yes	No	Yes	Read-only
Called using	Class::member / object	Class::function()	Object	Object

Static vs Const Keyword

Point	static	const
Meaning	Shared among all objects	Value cannot be changed
Memory	Single copy for class	Separate copy per object
Modification	Value can be modified	Value cannot be modified
Purpose	Memory sharing, counters	Data safety, fixed values

Polymorphism in C++

The word polymorphism means having many forms.
In C++, polymorphism concept can be applied to functions and operators.
A single function name can work differently in different situations.
Similarly, an operator works different when used in different context.



Point	Compile-Time Polymorphism	Runtime Polymorphism
Binding time	At compile time	At runtime
Achieved using	Function overloading, Operator overloading	Function overriding
Inheritance	Not required	Required
Virtual keyword	Not used	Required
Function call	Resolved early	Resolved late
Flexibility	Less flexible	More flexible

Static & Dynamic Binding

Polymorphism means one name, multiple forms. The overloaded member functions are selected for invoking by matching arguments, both type and number. This information is known to the compiler at the compile time and compiler is able to select the appropriate function for a particular call at the compile time itself. This is called **Early Binding** or **Static Binding** or **Static Linking**. Also known as **compile time polymorphism.** Early binding means that an object is bound to its function call at the compile time.

It would be nice if the appropriate member function could be selected while the program is running. This is known as **runtime polymorphism.** C++ supports a mechanism known as **virtual functio**n to achieve run time polymorphism.

At the runtime, when it is known what class objects are under consideration, the appropriate version of the function is invoked.

Since the function is linked with a particular class much later after the compilation, this process is termed as late binding. It is also known as **dynamic binding** because the selection of the appropriate function is done dynamically at run time.

Dynamic Binding in C++

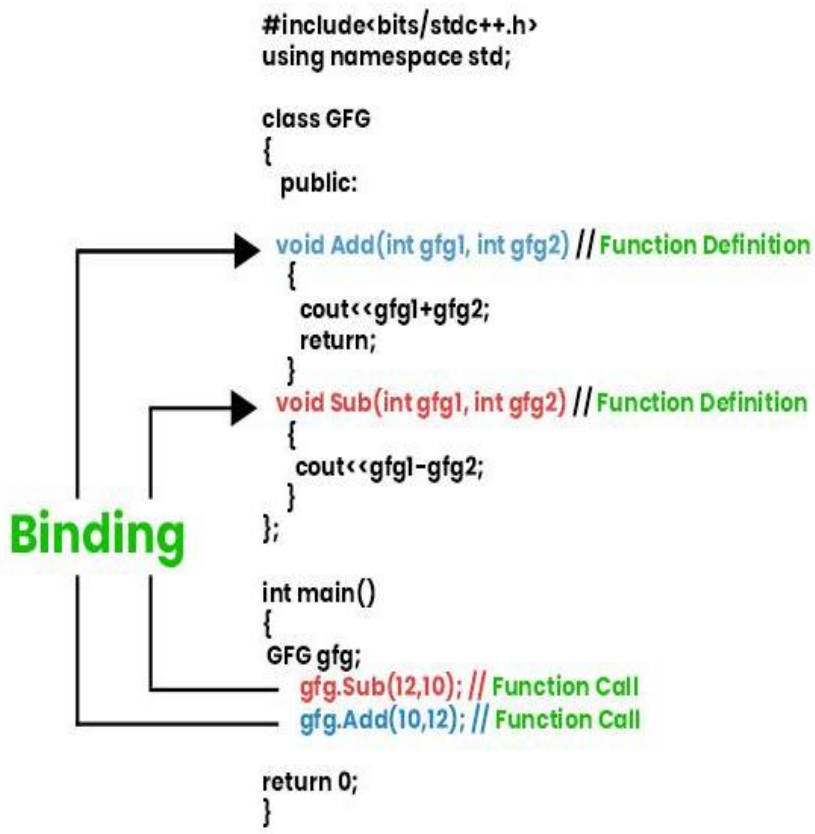
Dynamic binding in C++ is a practice of connecting the function calls with the function definitions by avoiding the issues with static binding, which occurred at build time. Because dynamic binding is flexible, it avoids the drawbacks of static binding, which connected the function call and definition at build time.

In simple terms, Dynamic binding is the connection between the function declaration and the function call.

So, choosing a certain function to run up until runtime is what is meant by the term Dynamic binding. A function will be invoked depending on the kind of item.

Usage of Dynamic Binding

It is also possible to use dynamic binding with a single function name to handle multiple objects. Debugging the code and errors is also made easier and complexity is reduced with the help of Dynamic Binding.



Dynamic Binding

Static Binding Vs Dynamic Binding

Static Binding	Dynamic Binding
It takes place at compile time which is referred to as early binding	It takes place at runtime so it is referred to as late binding
Execution of static binding is faster than dynamic binding because of all the information needed to call a function.	Execution of dynamic binding is slower than static binding because the function call is not resolved until runtime.
It takes place using normal function calls, operator overloading, and function overloading.	It takes place using virtual functions
Real objects never use static binding	Real objects use dynamic binding

VIRTUAL FUNCTIONS

Polymorphism refers to the property by which objects belonging to different classes are able to respond to the same message, but different forms. An essential requirement of polymorphism is therefore the ability to refer to objects without any regard to their classes.
When we use the same function name in both the base and derived classes, the function in the base class is declared as virtual using the keyword virtual preceding its normal declaration.
When a function is made virtual, C++ determines which function to use at runtime based on the type of object pointed to by the base pointer, rather than the type of the pointer. Thus, by making the base pointer to point to different objects, we can execute different versions of the virtual function.

```
     #include<iostream.h>
     class Base
    {
     public:
     void display()
     {
     cout<<"Display Base";
     }
  virtual void show()
    {
     cout<<"Show Base";
     }
    };
class Derived : public Base
  {

  public:
  void display()
   {
     cout<<"Display Derived";
   }
 void show()
  {
   cout<<"show derived";
  }
};
void main()
    {
     Base b;
     Derived d;
     Base *ptr;
  cout<<"ptr points to Base";
  ptr=&b;
```

```
 ptr->display(); //calls Base
 ptr->show(); //calls Base
  cout<<"ptr points to derived";
 ptr=&d;
 ptr->display(); //calls Base
 ptr->show(); //class Derived
}
```

Output:

```
ptr points to Base
Display Base
Show Base
ptr points to Derived
Display Base
Show Derived
```

When ptr is made to point to the object d, the statement ptr->display(); calls only the function associated with the Base i.e.. Base::display() where as the statement ptr->show(); calls the derived version of show(). This is because the function display() has not been made virtual in the Base class.

Rules for Virtual Functions:

When virtual functions are created for implementing late binding, observe some basic rules that satisfy the
compiler requirements.

1. The virtual functions must be members of some class.
2. They cannot be static members.
3. They are accessed by using object pointers.
4. A virtual function can be a friend of another class.
5. A virtual function in a base class must be defined, even though it may not be used.
6. The prototypes of the base class version of a virtual function and all the derived class versions must be identical. C++ considers them as overloaded functions, and the virtual function mechanism is ignored.
7. We cannot have virtual constructors, but we can have virtual destructors.
8. While a base pointer points to any type of the derived object, the reverse is not true. i.e. we cannot use a pointer to a derived class to access an object of the base class type.
9. When a base pointer points to a derived class, incrementing or decrementing it will not make it to point to the next object of the derived class. It is incremented or decremented only relative to its base type. Therefore we should not use this method to move the pointer to the next object.

10. If a virtual function is defined in the base class, it need not be necessarily redefined in the derived class. In such cases, calls will invoke the base function.

OPERATOR OVERLOADING

C++ has the ability to provide the operators with as special meaning for a data type. The mechanism of giving such special meanings to an operator is known as operator overloading. We can overload all the operators except the following:

- Class member access operator (“.” and “*”)
- Scope resolution operator “::”
- Size operator (sizeof)
- Conditional operator

To define an additional task to an operator, specify what it means in relation to the class to which the operator
is applied. This is done with the help of a special function, called operator function.

The process of overloading involves following steps:
1. Create a class that defines the data type that is to be used in the overloading operation.
2. Declare the operator function operator op() in the public part of the class. It may be a member function or a friend function.
3. Here op is the operator to be overloaded.
4. Define the operator function to implement the required operations.

```
return-type classname :: operator op(arglist)
{
Function body
}
complex complex::operator+(complex c)
{
complex t;
t.real=real+c.real;
t.img=img+c.img;
return t;
}
```

Concept of Operator Overloading

One of the unique features of C++ is Operator Overloading. Same operator is responding in a different manner.
For example operator + can be used as concatenate operator as well as additional operator. That is 2+3 means 5

(addition), where as "2"+"3" means 23 (concatenation).
Performing many actions with a single operator is operator overloading. We can assign a user defined function to an operator. We can change function of an operator, but it is not recommended to change the actual functions of operator. We can't create new operators using this operator overloading.
Operator overloading concept can be applied in following two major areas (Benefits)

1. Extension of usage of operators
2. Data conversions

Rules to be followed for operator overloading:-

1. Only existing operators can be overloaded.
2. Overloaded operators must have at least one operand that is of user defined operators
3. We cannot change basic meaning of an operator.
4. Overloaded operator must follow minimum characteristics that of original operator
5. When using binary operator overloading through member function, the left hand operand must be an object of relevant class

The number of arguments in the overloaded operator‟ s arguments list depends

- Operator function must be either member function or friend function.
- If operator function is a friend function then it will have one argument for unary operator & two arguments for binary operator
- If operator function is a member function then it will have Zero argument for unary operator & one argument for binary operator

Unary Operator Overloading

A unary operator means, an operator which works on single operand. For example, ++ is a unary operator, it takes single operand (c++). So, when overloading a unary operator, it takes no argument because object itself is considered as argument.

Syntax for Unary Operator (Inside a class)

```
return-type operator operatorsymbol( )
{
//body of the function
}
```

Ex:

```
void operator-()
{
Real = - real;
Img = - img;
}
```

Syntax for Un ary Operator (Outside a class)

```
return-type classname::operator operatorsymbol( )
{
//body of the function
}
```

Example 1:-

```
void operator++()
{
counter++;
}
```

Example 2:-

```
void complex::operator-()
{
real=-real;

img=-img;
}
```

The following simple program explains the concept of unary overloading.

```
#include < iostream.h >
#include < conio.h >
// Program Operator
Overloading class fact
        {
          int a;
             public:
                  fact ()
                  {
                  a=0;
                  }
                  fact (int i)
                  {
                  a=i;
                  }
```

```
    fact operator!()
                    {
                     int f=1,i;
                     fact t;
                    for (i=1;i<=a;i++)
                        {
                         f=f*i;
                        }
                     t.a=f;
                  return t;
                }
void display()
{
cout<<"The factorial "<< a;
}
};
void main()
{
int x;
cout<<"enter a number";
cin>>x;
fact s(x),p;
p=!s;
p.display();
}
```

Output for the above program:

Enter a number 5
The factorial of a given number 120

Explanation:

We have taken "!" as operator to overload. Here class name is fact. Constructor without parameters takes initially value of "x" as 0. Constructor with parameter takes the value of "x". We have create two objects one for doing the factorial and the other for return the factorial. Here number of parameter for an overloaded function is 0. Factorial is unary operator because it operates on one data item. Operator overloading find the factorial of the object. The display functions for printing the result.

Overloading Unary Operator -

Write a program to overload unary operator –

```
#include<iostream>
using namespace std;
class complex
{
float real,img;
public:
complex();
complex(float x, float y);
void display();
void operator-();
};
complex::complex()
{
real=0;img=0;
}
complex::complex(float x, float y)
{
real=x;
img=y;
}
void complex::display()
{
int imag=img;
if(img<0)
{
imag=-img;
cout<<real<<" -i"<<imag<<endl;
}
else
cout<<real<<" +i"<<img<<endl;
}
void complex::operator-()
{
real=-real;
img=-img;
}
int main()
{
complex c(1,-2);
c.display();
cout<<"After Unary - operation\n";
-c;
```

```
c.display();
}
```

Example 2:-

```
#include<iostream.h>
using namespace std;
class space
{
int x,y,z;
public:
void getdata(int a,int b,int c);
void display();
void operator-();
};
void space :: getdata(int a,int b,int c)
{
x=a;
y=b;
z=c;
}
void space :: display()
{
cout<<"x="<<x<<endl;
cout<<"y="<<y<<endl;
cout<<"z="<<z<<endl;
}
void space :: operator-()
{
x=-x;
y=-y;
z=-z;
}

int main()
{
space s;
s.getdata(10,-20,30);
s.display();
-s;
cout<<"after negation\n";
s.display();
}
```

Output:
x=10
y=-20
z=30
after negation
x=-10
y=20
z=-30

It is possible to overload a unary minus operator using a friend function as follows:
friend void operator-(space &s);

Unary minus operator using a friend function

```
#include<iostream.h>
#include<iostream.h>
using namespace std;
class space
{
int x,y,z;
public:
void g etdata(int a,int b,int c);
void display();
friend void operator-(space &);
};
void space :: getdata(int a,int b,int c)
{
x=a;
y=b;
z=c;
}
void space :: display()
{
cout<<x<<" "<<y<<" "<<z<<endl;
}
void operator-(space &s)
{
s.x=-s.x;
s.y=-s.y;
s.z=-s.z;
}
```

```
int main()
{
space S;
S.getdata(10,-20,30);
S.display();
-S;
cout<<"after negation\n";
S.display();
}
```

Output:
10 -20 30
after negation
-10 20 -30

Binary Operator Overloading

A binary operator means, an operator which works on two operands. For example, + is an binary operator, it takes single operand (c+d). So, when overloading an binary operator, it takes one argument (one is object itself and other one is passed argument).
Syntax for Binary Operator (Inside a class)

```
return-type operator operatorsymbol(argument)
{
//body of the function
}
```

Syntax for Binary Operator definition (Outside a class)

```
return-type classname::operator operatorsymbol(argument)
{
//body of the function
}
```

Example

```
complex operator+(complex s)
{
complex t;
t.real=real+s.real;
t.img=img+s.img;
```

```
return t;
}
```

The following program explains binary operator overloading:

```
#include < iostream.h >
#include < conio.h >
class sum
{
int a;
public:
sum()
{
a=0;
}
sum(int i)
{
a=i;
}
sum operator+(sum p1)
{
sum t;
t.a=a+p1.a;
return t;
}
void main ()

{
cout<<"Enter Two Numbers:"
int a,b;
cin>>a>>b;
sum x(a),y(b),z;
z.display();
z=x+y;
cout<<"after applying operator \n";
z.display();
getch();
}
```

Output:

Enter two numbers 5 6
After applying operator
The sum of two numbers 11

Explanation: The class name is “sum”. We have created three objects two for to do the sum and the other for returning the sum. “ +“ is a binary operator operates on members of two objects and returns the result which is member of a object. Here number of parameters is 1. The sum is displayed in display function.

Write a program to over load arithmetic operators on complex numbers using member function

```
#include<iostream.h>
class complex
{
float real,img;
public:
complex(){ }
complex(float x, float y)
{
real=x;
img=y;
}
complex operator+(complex c)
void display();
};
complex complex::operator+(complex c)
{
complex temp;
temp.real=real+c.real;
temp.img=img+c.img;
return temp;
```

```
}
void complex::display()
{
int imag=img;
If(img<0)
{
imag=-imag;

cout<<real<<”-i”<<imag;
}
else
cout<<real<<”+i”<<img;
}
int main()
{
complex c1,c2,c3;
c1=complex(2.5,3.5);
c2=complex(1.6,2.7);
c3=c1+c2;
c3.display();
return 0;
}
```

Overloading Binary Operators Using Friends

1. Replace the member function declaration by the friend function declaration in

the class friend complex operator+(complex, complex)

2. Redefine the operator function as follows:

```
    complex op erator+(complex a, complex b)
  {
    return complex((a.x+b.x),(a.y+b.y));
   }
```

Write a program to over load arithmetic operators on complex numbers using friend

Function.

```
#include<iostream.h>
class complex
{
float real,img;
public:
complex(){ }
complex(float x, float y)
```

```
{
real=x;
img=y;
}
friend complex operator+(complex);
void display();
};
complex operator+(complex c1, complex c2)
{
complex temp;
temp.real=c1.real+c2.real;
temp.img=c1.img+c2.img;
return temp;
}
void complex::display()

{
If(img<0)
{
img=-img;
cout<<real<<"-i"<<img;
}
else
cout<<real<<"+i"<<img;
}
int main()
{
complex c1,c2,c3;
c1=complex(2.5,3.5);
c2=complex(1.6,2.7);
c3=c1+c2;
c3.display();
return 0;
}
```